EXP NO: 6	BUILDING A SIMPLE NEURAL NETWORK

AIM:

To build, train and evaluate a simple feed-forward neural network (Multi-Layer Perceptron) for classifying handwritten digits.

ALGORITHM:

1. Import the required libraries (numpy, matplotlib, seaborn, scikit-learn).
2. Load the Digits dataset (8x8 images flattened to 64 features, 10 classes).
3. Scale the features using StandardScaler and split into training and testing sets.
4. Build the network using MLPClassifier with hidden layers of 64 and 32 neurons, ReLU activation and the Adam optimiser.
5. Train the network using fit() and display the architecture and number of parameters.
6. Predict the test data and evaluate using accuracy, classification report and confusion matrix.
7. Plot the training loss curve and display some sample predictions.

CODE:

```
# 1. Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report
import seaborn as sns

# 2. Load dataset (8x8 handwritten digit images, 10 classes)
digits = load_digits()
X, y = digits.data, digits.target
print("Input shape:", X.shape, " Classes:", np.unique(y))

# 3. Scale and split
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
# 4. Build the neural network: 64 -> 64 -> 32 -> 10
```

```
mlp = MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu',
                    solver='adam', learning_rate_init=0.001,
                    max_iter=200, random_state=42)

# 5. Train
mlp.fit(X_train, y_train)

# 6. Network summary
layer_sizes = [X.shape[1]] + list(mlp.hidden_layer_sizes) + [mlp.n_outputs_]
params = sum(w.size for w in mlp.coefs_) + sum(b.size for b in mlp.intercepts_)
print("\nArchitecture (neurons per layer):", layer_sizes)
print("Total trainable parameters:", params)
print("Iterations run:", mlp.n_iter_, "  Final training loss:",
    round(mlp.loss_, 5))

# 7. Evaluate
y_pred = mlp.predict(X_test)
print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# 8. Training loss curve and confusion matrix
plt.figure(figsize=(11, 4))
plt.subplot(1, 2, 1)
plt.plot(mlp.loss_curve_)
plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("Training Loss Curve")
plt.subplot(1, 2, 2)
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix"); plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.tight_layout()
plt.show()

# 9. Sample predictions
raw = load_digits().images
_, idx_test = train_test_split(np.arange(len(y)), test_size=0.2,
    random_state=42, stratify=y)
plt.figure(figsize=(10, 2.5))
for i in range(8):
    plt.subplot(1, 8, i + 1)
    plt.imshow(raw[idx_test[i]], cmap='gray'); plt.axis('off')
    plt.title(f"P:{y_pred[i]}\nA:{y_test.iloc[i] if hasattr(y_test,'iloc') else
        y_test[i]}")
plt.suptitle("Sample predictions (P = predicted, A = actual)")
plt.show()
```

OUTPUT:

```
Input shape: (1797, 64)  Classes: [0 1 2 3 4 5 6 7 8 9]

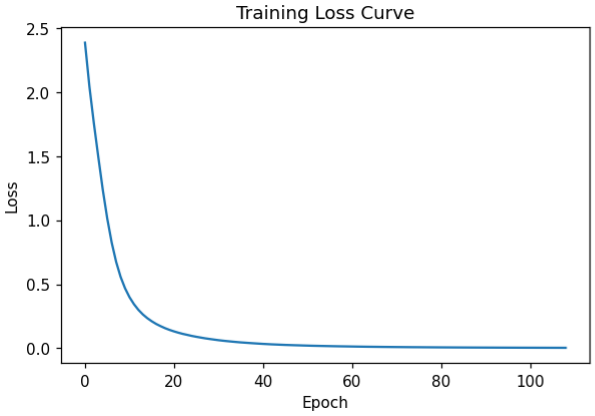
Architecture (neurons per layer): [64, 64, 32, 10]
Total trainable parameters: 6570
Iterations run: 109   Final training loss: 0.00339

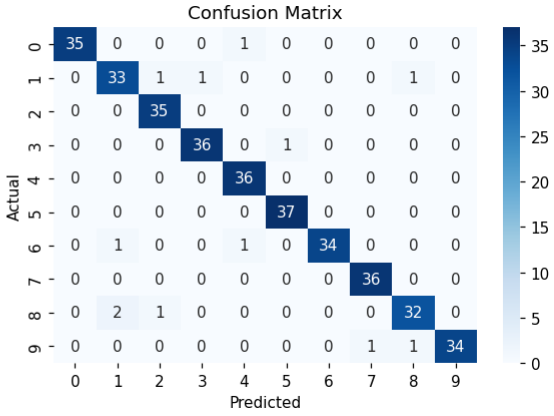
Test Accuracy: 0.9666666666666667

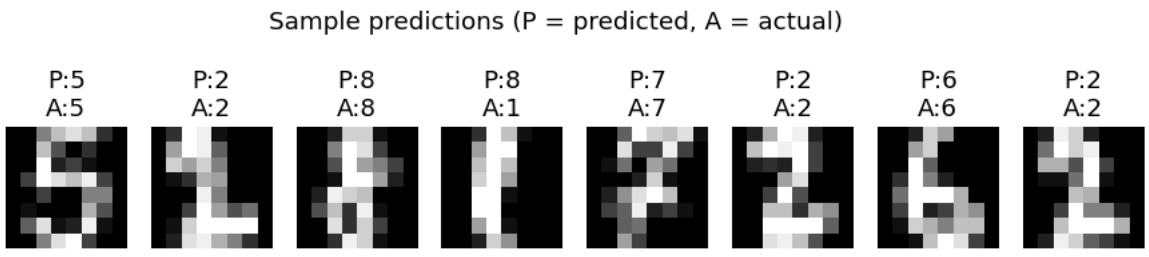
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.97      0.99        36
           1       0.92      0.92      0.92        36
           2       0.95      1.00      0.97        35
           3       0.97      0.97      0.97        37
           4       0.95      1.00      0.97        36
           5       0.97      1.00      0.99        37
           6       1.00      0.94      0.97        36
           7       0.97      1.00      0.99        36
           8       0.94      0.91      0.93        35
           9       1.00      0.94      0.97        36

    accuracy                           0.97       360
   macro avg       0.97      0.97      0.97       360
weighted avg       0.97      0.97      0.97       360
```







RESULT:

A simple neural network with the architecture 64-64-32-10 (6,570 trainable parameters) was successfully built and trained. The training loss decreased steadily and the model achieved a test accuracy of 96.67% on the handwritten digits dataset.